FFNN untuk Prediksi Penempatan Kerja

Shanice Feodora Tjahjono **Muhammad Fathur Rizky** **Ahmad Wicaksono**
Kelompok `TorchIsAllYouNeed`, Program Studi Teknik Informatika, Institut Teknologi Bandung
`{13523097,13523105,13523121}@std.stei.itb.ac.id`

Abstrak

Laporan ini membahas pembuatan *feedforward neural network* (FFNN) dari awal dengan `NumPy` untuk memprediksi status penempatan kerja pada dataset `global_student_placement_and_salary`. Tujuan kami bukan hanya membuat model yang bisa dilatih, tetapi juga menyusun mini-framework yang cukup lengkap. Framework ini mencakup *automatic differentiation* berbasis graph komputasi dinamis, abstraksi modul jaringan, fungsi aktivasi dan *loss*, beberapa skema inisialisasi bobot, regularisasi L1/L2, optimizer `SGD` dan `Adam`, serta `RMSNorm`. Dari hasil eksperimen, terlihat bahwa performa model sangat dipengaruhi oleh kapasitas jaringan, inisialisasi, aktivasi, dan cara optimisasi yang dipakai. Konfigurasi terbaik pada studi kapasitas mencapai akurasi validasi 0.7520. Pada studi inisialisasi, `zero` tidak memberi titik awal pelatihan yang baik (akurasi 0.6155; *validation loss* 0.666224), sedangkan Xavier memberi hasil terbaik di antara tiga skema yang diuji (*validation loss* 0.488897). Penambahan `RMSNorm` juga menaikkan akurasi dari 0.7395 menjadi 0.7480 walaupun tidak selalu menurunkan *loss*. Selain itu, implementasi kami juga lebih baik daripada baseline `sklearn MLPClassifier` pada setup yang sebanding (akurasi 0.7520 vs. 0.6975). Secara umum, hasil ini menunjukkan bahwa pilihan desain sistem dan detail implementasi numerik cukup berpengaruh terhadap stabilitas pelatihan dan kemampuan generalisasi model.

1 Deskripsi Persoalan

Dataset `global_student_placement_and_salary` berisi informasi akademik mahasiswa, pengalaman *internship*, serta konteks institusi dan negara asal, dengan target `placement_status`. Dalam laporan ini, persoalan tersebut dipandang sebagai klasifikasi biner: apakah seorang mahasiswa termasuk *Placed* atau *Not Placed*. Artinya, model harus mempelajari hubungan yang tidak selalu linear antara fitur-fitur numerik dan kategorikal, lalu mengubahnya menjadi probabilitas penempatan kerja.

2 Pembahasan

Implementasi pada proyek ini sengaja dibagi menjadi tiga lapisan utama. Lapisan pertama adalah *autodiff core* pada kelas `Tensor` yang mengurus operasi diferensiabel dan aliran gradien. Lapisan kedua adalah abstraksi `nn.Module` yang menjadi dasar untuk komponen seperti layer, fungsi aktivasi, normalisasi, dan *loss*. Lapisan ketiga adalah `optim` yang mengurus aturan update parameter. Pembagian ini membantu proses eksperimen karena arsitektur model bisa diubah, optimizer bisa diganti, dan fungsi aktivasi bisa dibandingkan tanpa harus mengubah mesin gradien utamanya. Susunan seperti ini juga sejalan dengan cara kerja sistem autodiff modern yang memisahkan graph diferensiasi, struktur model, dan proses optimisasi [14, 20].

3 Penjelasan Implementasi

3.1 Arsitektur Implementasi

Arsitektur sistem dibangun di atas tiga lapisan yang saling melengkapi. Lapisan pertama adalah *autodiff core* pada kelas `Tensor`. Di sini, satu objek menyimpan nilai numerik, gradien, parent pada graph komputasi, dan aturan turunan lokal. Lapisan kedua adalah abstraksi `Module` yang menjadi dasar untuk komponen model seperti `Linear`, `FFNN`, `Sequential`, kelas-kelas aktivasi, kelas-kelas *loss*, dan `RMSNorm`. Lapisan ketiga adalah keluarga `Optimizer` yang memisahkan logika update parameter dari graph diferensiasi. Dengan pembagian seperti ini, tiap bagian punya tugas yang jelas dan perubahan pada satu bagian tidak langsung merusak bagian lain [14, 20].

Dari sisi implementasi, susunan ini membuat alurnya lebih mudah diikuti. Data lebih dulu dibungkus menjadi `Tensor`, lalu diproses oleh rangkaian `Module`, menghasilkan *loss* skalar, dan akhirnya parameternya diperbarui oleh `Optimizer`. Pada level model, `Linear` dipakai untuk operasi affine dasar, `Sequential` berperan sebagai kompositor modul umum, sedangkan `FFNN` menjadi pembungkus yang lebih praktis untuk eksperimen utama. Pada level data, `ArrayDataset` dan `DataLoader` menangani batching, pengacakan, dan reproduksibilitas eksperimen. Selain API berbasis kelas, sistem ini juga punya lapisan fungsional stateless untuk aktivasi dan *loss* sehingga perhitungan numerik bisa diuji terpisah dari struktur model.

Tiga mekanisme API utama

Implementasi dibagi menjadi tiga mekanisme API. Mekanisme pertama adalah backend `Tensor` yang menangani operasi numerik dasar, perekaman graph, dan `backward()`. Mekanisme kedua adalah API layer `nn` yang berisi komponen stateful seperti `Linear`, `RMSNorm`, fungsi aktivasi, dan kelas-kelas *loss*. Mekanisme ketiga adalah fungsi stateless pada namespace `torchlike.functional` yang dipakai untuk komputasi langsung berbasis NumPy atau `Tensor` ketika diperlukan validasi numerik atau komposisi yang lebih ringan.

Method `Module.parameters()` menjadi penghubung utama antara model dan proses optimisasi karena method ini mengumpulkan seluruh parameter trainable secara rekursif, termasuk yang tersimpan di dalam `list`, `tuple`, `dict`, atau submodule. Jadi, saat blok jaringan baru ditambahkan, pengembang cukup mendefinisikan atribut `Tensor(requires_grad=True)` pada turunan `Module` dan parameternya akan ikut terdeteksi tanpa registrasi manual. Mekanisme ini sederhana tetapi penting karena menjadi dasar untuk pembahasan *forward pass*, inisialisasi, aktivasi, *loss*, autodiff, dan optimisasi.

3.2 Struktur Kode Utama

Bagian ini berfokus pada bentuk implementasi nyatanya. Setiap cuplikan kode diikuti penjelasan singkat agar pembaca bisa melihat dengan jelas bagaimana ide desain di bagian sebelumnya diterjemahkan menjadi kode.

Kelas dasar `Module`

```
class Module:
    def forward(self, *args, **kwargs):
        raise NotImplementedError

    def __call__(self, *args, **kwargs):
        return self.forward(*args, **kwargs)

    def parameters(self):
        out, seen = [], set()
        for value in self.__dict__.values():
            self._collect_parameters(value, out, seen)
        return out
```

Kelas ini berfungsi sebagai kontrak dasar bagi semua komponen jaringan. Method `forward()` wajib diimplementasikan oleh kelas turunannya. Sementara itu, `parameters()` bertugas mengumpulkan semua `Tensor` trainable secara rekursif. Dengan desain seperti ini, optimizer tidak perlu mengetahui struktur internal model secara detail; ia cukup menerima daftar parameter yang sudah terkumpul.

Alur reverse-mode pada `Tensor.backward`

```
def backward(self, grad=None):
    if grad is None:
        grad = np.ones_like(self.data)

    self.grad = self.grad + grad if self.grad is not None else grad
    topo, visited = [], set()

    def build(node):
        if id(node) in visited:
            return
        visited.add(id(node))
        for child in node._prev:
            build(child)
        topo.append(node)

    build(self)
    for node in reversed(topo):
        node._backward()
```

Implementasi autodiff memakai graph dinamis. Fungsi `build()` terlebih dahulu menyusun urutan topologis node-node yang relevan, lalu traversal terbalik menjalankan *chain rule*. Setiap operasi tensor menyimpan aturan turunan lokalnya sendiri di `_backward`. Karena itu, engine gradien tidak perlu mengetahui rumus khusus untuk seluruh model; ia cukup mengeksekusi aturan lokal setiap node secara berurutan.

Layer `Linear`**,** `Sequential`**, dan** `FFNN`

```
class Linear(Module):
    def __init__(self, in_features, out_features, init="uniform", seed=None):
        self.weight = Tensor(_initialize((in_features, out_features), ...),
                             requires_grad=True)
        self.bias = Tensor(_initialize((1, out_features), ...),
                           requires_grad=True)

    def forward(self, x: Tensor) -> Tensor:
```

```
        return F.linear(x, self.weight, self.bias)

class Sequential(Module):
    def __init__(self, layers):
        self.layers = list(layers)

    def forward(self, x: Tensor) -> Tensor:
        out = x
        for layer in self.layers:
            out = layer(out)
        return out

class FFNN(Module):
    def __init__(self, layer_sizes, activations=None, init="uniform", seed=None):
        layers = []
        ...
        self.network = Sequential(layers)

    def forward(self, x: Tensor) -> Tensor:
        return self.network(x)
```

`Linear` menangani inisialisasi bobot sekaligus operasi affine dasar. `Sequential` mengatur alur eksekusi layer secara deklaratif dari awal sampai akhir. Di atasnya, `FFNN` menyusun beberapa layer linear dan aktivasi menjadi arsitektur yang siap dipakai di eksperimen. Jadi, pada notebook terbaru, sebagian besar studi memakai `FFNN` sebagai antarmuka utama, sedangkan `Sequential` tetap dipakai ketika susunan layer perlu dirakit lebih manual.

Catatan transisi optimizer

Implementasi detail update bobot sengaja dipusatkan pada Subbagian 3.11 agar pembahasan tidak berulang. Bagian ini hanya memperlihatkan struktur model dan loop pelatihan tingkat tinggi.

3.3 Penyusunan Berkas Kode

Subbagian ini menjelaskan bagaimana kode disusun supaya alur implementasinya mudah diikuti, mulai dari level folder hingga loop pelatihan.

Struktur direktori inti

```
src/
  main.py
  torchlike/
    tensor.py
    data.py
    nn/
      module.py
      linear.py
      sequential.py
      activations.py
      normalization.py
      loss.py
    functional/
      activations/
      losses/
      layers/
    optim/
      base.py
      sgd.py
      adam.py
tests/
  test_tensor.py
  test_nn.py
  test_functional.py
  test_optim.py
```

Struktur ini memisahkan tanggung jawab tiap bagian secara cukup bersih. Folder `torchlike` berisi library inti, sedangkan `main.py` berfungsi sebagai entry point eksperimen. Dengan susunan seperti ini, perubahan pada level algoritma tidak langsung mengganggu pipeline data atau kode eksperimen.

Alur pelatihan pada `main.py`

```
x_train, x_val, y_train, y_val = load_dataset_features(csv_path)
train_loader = DataLoader(ArrayDataset(x_train, y_train), batch_size=256, shuffle=
    True)

model = build_binary_model(x_train.shape[1], use_rmsnorm=True, init="he")
criterion = nn.BCEWithLogitsLoss()
optimizer = create_optimizer("adam", model)

for xb, yb in train_loader:
    logits = model(xb)
    loss = criterion(logits, yb)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Potongan ini menunjukkan urutan minimum yang dipakai di seluruh eksperimen: siapkan data, bangun model, hitung *loss*, lakukan *backward*, lalu perbarui parameter. Karena polanya konsisten, studi ablation dapat dilakukan cukup dengan mengganti pembangun model atau konfigurasi optimizer tanpa menulis ulang loop pelatihan.

3.4 Forward Propagation

Untuk FFNN dengan L layer, *forward pass* dituliskan sebagai:

$$\mathbf{z}^{(l)} = \mathbf{a}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \phi\left(\mathbf{z}^{(l)}\right). \tag{1}$$

Di sini, $\mathbf{a}^{(0)} = \mathbf{x}$ menyatakan vektor input. Jadi, untuk setiap $l = 1, \ldots, L$, keluaran suatu layer menjadi masukan bagi layer berikutnya secara eksplisit. Pada implementasi, layer terakhir dibiarkan linear, artinya tidak ada fungsi aktivasi tambahan setelahnya. Untuk klasifikasi biner, keluaran akhir ini berbentuk logit skalar per sampel. Jika yang dibutuhkan adalah probabilitas, logit tersebut baru dipetakan dengan sigmoid saat evaluasi. Namun, jika *loss* yang dipakai adalah `BCEWithLogitsLoss`, sigmoid eksplisit pada *forward pass* tidak perlu lagi dituliskan karena sudah digabungkan secara stabil secara numerik [22]:

$$\mathcal{L}(z, y) = \max(z, 0) - zy + \log\left(1 + e^{-|z|}\right), \tag{2}$$

bentuk ini dipakai untuk menghindari *overflow* saat $|z|$ besar. Pada level implementasi, peta affine $\mathbf{a}^{(l-1)} \mapsto \mathbf{z}^{(l)}$ direalisasikan oleh kelas `Linear` dengan bobot berukuran $(d_{\text{in}}, d_{\text{out}})$ dan bias opsional berukuran $(1, d_{\text{out}})$. Konvensi penyimpanan ini mengikuti bentuk komputasi maju $\mathbf{y} = \mathbf{x}W + \mathbf{b}$ sehingga sedikit berbeda dari konvensi internal `nn.Linear` pada PyTorch yang umum memakai $(d_{\text{out}}, d_{\text{in}})$. Setelah itu, layer-layer tersebut dirangkai oleh `Sequential` melalui antarmuka `__call__()` dan `forward()`, lalu dibungkus oleh kelas `FFNN` agar konfigurasi *layer size* dan aktivasi bisa ditulis lebih ringkas pada eksperimen.

Cuplikan implementasi alur *forward*

```
def linear(x, weight, bias=None):
    x_t = x if isinstance(x, Tensor) else Tensor(x, requires_grad=False)
    w_t = weight if isinstance(weight, Tensor) else Tensor(weight, requires_grad=
    False)
    output = x_t @ w_t
    if bias is not None:
        b_t = bias if isinstance(bias, Tensor) else Tensor(bias, requires_grad=False
    )
        output = output + b_t
    return output
```

```
class Linear(Module):
    def forward(self, x: Tensor) -> Tensor:
        return F.linear(x, self.weight, self.bias)

class Sequential(Module):
    def forward(self, x: Tensor) -> Tensor:
        out = x
        for layer in self.layers:
            out = layer(out)
        return out
```

Cuplikan ini memperlihatkan bahwa transformasi affine dihitung di `F.linear`, lalu setiap layer dipanggil berurutan oleh `Sequential.forward`. Kelas `FFNN` memanfaatkan kompositor ini di belakang layar sehingga eksperimen bisa fokus pada susunan dimensi dan aktivasi tanpa merakit seluruh layer satu per satu.

Setelah alur *forward* ini jelas, pertanyaan berikutnya adalah bagaimana memilih parameter awal agar sinyal aktivasi dan gradien tidak langsung rusak sejak iterasi pertama.

3.5 Inisialisasi Bobot

Inisialisasi bobot menentukan skala awal parameter sebelum pelatihan dimulai. Tujuan praktisnya sederhana tetapi penting: pada beberapa iterasi pertama, aktivasi dan gradien sebaiknya tidak langsung meledak atau justru menghilang [4, 5, 1]. Pada layer affine

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}, \tag{3}$$

skala bobot terutama ditentukan oleh banyaknya koneksi masuk (*fan_in*) dan koneksi keluar (*fan_out*).

Skema *Xavier* umumnya dipakai ketika aktivasi relatif simetris di sekitar nol. Intuisi utamanya adalah menjaga agar skala sinyal pada *forward pass* dan *backward pass* tetap seimbang dengan mempertimbangkan *fan_in* dan *fan_out* sekaligus. Dengan cara ini, ketika dimensi layer membesar, rentang bobot awal otomatis diperkecil agar keluaran awal tetap berada pada skala yang masuk akal.

Skema *He* lebih ditujukan untuk aktivasi bertipe rectifier seperti ReLU dan masih relevan untuk aktivasi non-saturasi yang sejenis. Karena hanya *fan_in* yang dipakai, skala awal dari He cenderung sedikit lebih agresif daripada Xavier pada layer yang lebar.

Supaya hubungan antara teori dan implementasi lebih jelas, dua besaran skala utama yang dipakai di kode adalah

$$a_{\text{xavier}} = g\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \qquad \sigma_{\text{he}} = \sqrt{\frac{2}{d_{\text{in}}}}. \tag{4}$$

Besaran a_{xavier} dipakai sebagai batas pada Xavier uniform, sedangkan σ_{he} dipakai sebagai simpangan baku pada He normal.

Implementasi fungsi `_initialize` untuk Xavier dan He

```
fan_in = shape[0]
fan_out = shape[1] if len(shape) > 1 else shape[0]

if method in ("xavier", "xavier_uniform", "xavier_normal"):
    gain = float(params.get("gain", 1.0))
    if method == "xavier_normal":
        std = gain * np.sqrt(2.0 / (fan_in + fan_out))
        return rng.normal(0.0, std, size=shape).astype(np.float64)
    limit = gain * np.sqrt(6.0 / (fan_in + fan_out))
    return rng.uniform(-limit, limit, size=shape).astype(np.float64)

if method in ("he", "he_uniform", "he_normal"):
    if method == "he_uniform":
        limit = np.sqrt(6.0 / fan_in)
        return rng.uniform(-limit, limit, size=shape).astype(np.float64)
    std = np.sqrt(2.0 / fan_in)
```

```
    return rng.normal(0.0, std, size=shape).astype(np.float64)
```

Potongan ini menunjukkan penerjemahan langsung dari rumus matematis ke operasi NumPy. Cabang Xavier memakai `fan_in + fan_out`, sedangkan cabang He hanya memakai `fan_in`, sesuai asumsi yang melatarbelakangi inisialisasi untuk aktivasi rectifier.

Inisialisasi bobot dan bias di konstruktor `Linear`

```
self.weight = Tensor(
    _initialize((self.in_features, self.out_features),
                method=init, params=self.init_params, rng=rng),
    requires_grad=True,
)
self.bias = Tensor(
    _initialize((1, self.out_features),
                method=("zero" if init.lower().strip() in
                        ("xavier", "xavier_uniform", "xavier_normal",
                         "he", "he_uniform", "he_normal") else init),
                params={} if init.lower().strip() in
                       ("xavier", "xavier_uniform", "xavier_normal",
                        "he", "he_uniform", "he_normal")
                       else self.init_params,
                rng=rng),
    requires_grad=True,
) if self.use_bias else None
```

Desain ini memastikan bobot benar-benar mengikuti skema inisialisasi yang dipilih. Pada saat yang sama, bias di-set ke nol untuk keluarga Xavier dan He agar offset awal tidak terlalu mendominasi nilai praaktivasi. Skema `zero` tetap disediakan sebagai *sanity check*, meskipun pada hidden layer skema ini memang gagal memecah simetri antarneuron.

3.6 Fungsi Aktivasi dan Turunan

Fungsi aktivasi dibangun dalam dua lapisan implementasi. Lapisan pertama berupa wrapper berbasis `Module` agar aktivasi bisa langsung disusun di dalam `Sequential` maupun `FFNN`. Lapisan kedua berupa fungsi stateless pada namespace `torchlike.functional` yang menjalankan operasi numeriknya.

Wrapper aktivasi pada namespace `nn`

```
class ReLU(Module):
    def forward(self, x: Tensor) -> Tensor:
        return F.relu(x)

class Softmax(Module):
    def __init__(self, axis: int = -1) -> None:
        self.axis = axis

    def forward(self, x: Tensor) -> Tensor:
        return F.softmax(x, axis=self.axis)

class GELU(Module):
    def __init__(self, alpha: float = 1.702) -> None:
        self.alpha = alpha

    def forward(self, x: Tensor) -> Tensor:
        return F.gelu(x, alpha=self.alpha)
```

Pola ini membuat antarmuka aktivasi konsisten dengan layer lain. Saat eksperimen mengganti ReLU, GELU, SELU, atau Tanh, struktur model tidak perlu diubah karena semuanya mengikuti kontrak `Module` yang sama.

Softmax stabil numerik

```
def softmax(x: np.ndarray | Tensor, axis: int = -1) -> np.ndarray | Tensor:
    if isinstance(x, Tensor):
        return x.softmax(axis=axis)
    shifted = x - np.max(x, axis=axis, keepdims=True)
    exp_values = np.exp(shifted)
    return exp_values / np.sum(exp_values, axis=axis, keepdims=True)
```

Baris `shifted = x - max(x)` berfungsi mencegah *overflow* ketika nilai logit sangat besar. Untuk input bertipe `Tensor`, perhitungan gradiennya ditangani oleh engine autodiff sehingga implementasi aktivasi tetap ringkas di level kode.

Turunan lokal yang dipakai pada backward untuk aktivasi utama adalah

$$\frac{d}{dx}\mathrm{ReLU}(x) = \mathbb{I}(x > 0), \quad \frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x)), \quad \frac{d}{dx}\tanh(x) = 1 - \tanh^2(x). \tag{5}$$

Untuk aktivasi bonus, implementasi memakai aproksimasi

$$\mathrm{GELU}(x) \approx x\,\sigma(1.702x), \tag{6}$$

serta formulasi *piecewise* standar untuk SELU [6, 7]. Dengan cakupan ini, eksperimen bisa membandingkan aktivasi yang cenderung saturatif dan yang non-saturatif dalam kerangka kode yang sama.

3.7 Mitigasi *Vanishing Gradient*

Bagian ini berfokus pada langkah-langkah praktis yang benar-benar dipakai di kode. Mitigasi utama yang digunakan adalah inisialisasi Xavier/He, aktivasi non-saturasi seperti GELU atau SELU, dan opsi RMSNorm pada hidden layer. Tiga komponen ini dipilih karena relatif sederhana, tetapi efektif untuk membantu menjaga skala gradien selama pelatihan.

Penyusunan model untuk menekan *vanishing gradient*

```
model = nn.FFNN(
    [input_dim, 32, 32, 32, 1],
    activations=["gelu", "selu", "gelu", None],
    init="he",
    seed=123,
)
```

Inisialisasi `he` dipilih pada layer linear karena cocok untuk aktivasi yang sifatnya rectifier-like. Aktivasi GELU dan SELU dipakai pada hidden layer agar gradien tidak cepat jenuh. Pada eksperimen utama, susunan seperti ini dibangun lewat kelas `FFNN`. Untuk eksperimen RMSNorm, penyusunan layer dilakukan lebih manual dengan `Sequential` agar `RMSNorm` bisa disisipkan di antara layer-layer linear dan aktivasi.

Analisis ringkas Untuk memantau pelemahan gradien antar layer digunakan rasio

$$q = \frac{r_1}{r_L}, \qquad r_l = \frac{\|\nabla_{W^{(l)}}\mathcal{L}\|_F}{\|W^{(l)}\|_F + \epsilon}. \tag{7}$$

Nilai q yang sangat kecil menunjukkan bahwa gradien di layer awal jauh lebih lemah daripada gradien di layer akhir. Dalam laporan ini, nilai tersebut tidak dibaca sendirian, tetapi dipadukan dengan plot distribusi gradien dari kelas `Sequential` agar interpretasinya lebih informatif.

3.8 Fungsi Loss dan Mekanisme Numerik

Fungsi *loss* memetakan keluaran model dan target ke sebuah skalar yang kemudian dijadikan objective optimisasi. Dari sudut pandang statistik, pilihan *loss* sekaligus mencerminkan model probabilistik implisit yang diasumsikan terhadap data [1]. Karena itu, subbagian ini tidak hanya membahas rumus objective, tetapi juga menjelaskan stabilitas numerik dan bagaimana gradien mengalir dari kelas

loss menuju backend autodiff `Tensor`. Pada implementasi, objective tersebut diekspos melalui kelas `MSELoss`, `BCELoss`, `CrossEntropyLoss`, dan `BCEWithLogitsLoss`.

Implementasi `Tensor.backward()` pada backend autodiff

```
def backward(self, grad: np.ndarray | None = None) -> None:
    if not self.requires_grad:
        return
    if grad is None:
        if self.data.size != 1:
            raise RuntimeError("grad must be specified for non-scalar tensor")
        grad = np.ones_like(self.data)

    self.grad = self.grad + grad if self.grad is not None else grad

    topo, visited = [], set()

    def build(node: Tensor) -> None:
        node_id = id(node)
        if node_id in visited:
            return
        visited.add(node_id)
        for child in node._prev:
            build(child)
        topo.append(node)

    build(self)
    for node in reversed(topo):
        node._backward()
```

Semua *loss* pada API layer `nn` pada akhirnya memanggil mekanisme `backward()` ini untuk menyebarkan gradien dari *loss* skalar ke setiap parameter trainable. Pemisahan ini membuat antarmuka `nn` tetap ringkas, sementara detail diferensiasi tetap terkonsentrasi di backend `Tensor`.

MSE sebagai negative log-likelihood Gaussian. Untuk prediksi $\hat{y}$ dan target y, MSE dihitung sebagai

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2. \tag{8}$$

Loss ini muncul secara alamiah jika diasumsikan

$$y_i = \hat{y}_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2). \tag{9}$$

Negatif log-likelihood Gaussian, dengan mengabaikan konstanta yang tidak bergantung pada parameter, adalah

$$-\log p(\mathbf{y} \mid \hat{\mathbf{y}}) = \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 + \text{konstanta}. \tag{10}$$

Jadi meminimalkan MSE ekuivalen dengan maximum likelihood di bawah noise Gaussian homogen. Gradien terhadap prediksi adalah

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i), \tag{11}$$

yang linier terhadap residual sehingga MSE sensitif terhadap outlier.

Implementasi `MSELoss`

```
class MSELoss(_Loss):
    def forward(self, pred, target) -> Tensor:
        p = _ensure_tensor(pred)
        t = _ensure_tensor(target)
        diff = p - t
        return (diff * diff).mean()
```

Implementasi ini sengaja dibuat ringkas agar fokus utamanya tetap pada residual kuadrat rata-rata. Seluruh operasi berjalan di atas graph `Tensor` sehingga gradien akan tersedia secara otomatis ketika `backward()` dipanggil.

Backward lokal yang dipakai `MSELoss`

```
def __mul__(self, other):
    other_t = self._coerce(other)
    out = Tensor(self.data * other_t.data, ...)

    def _backward():
        if self.requires_grad and self.grad is not None:
            self.grad += _unbroadcast(out.grad * other_t.data, self.shape)
        if other_t.requires_grad and other_t.grad is not None:
            other_t.grad += _unbroadcast(out.grad * self.data, other_t.shape)

def mean(self, axis=None, keepdims=False):
    if axis is None:
        denom = self.data.size
    elif isinstance(axis, tuple):
        denom = int(np.prod([self.data.shape[a] for a in axis]))
    else:
        denom = self.data.shape[axis]
    return self.sum(axis=axis, keepdims=keepdims) / denom
```

Pada MSE, operasi kuncinya adalah pengkuadratan elemen demi elemen lalu reduksi `mean`. Cuplikan ini memperlihatkan bagaimana gradien mengalir melalui aturan hasil kali dan pembagian oleh jumlah elemen saat `mean` dihitung.

Binary Cross-Entropy sebagai negative log-likelihood Bernoulli. Untuk klasifikasi biner dengan probabilitas prediksi $\hat{y}_i \in (0, 1)$ dan target $y_i \in \{0, 1\}$, BCE didefinisikan sebagai

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]. \tag{12}$$

Rumus ini adalah negative log-likelihood dari distribusi Bernoulli

$$p(y_i \mid \hat{y}_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1 - y_i}. \tag{13}$$

Mengambil log, menjumlahkan untuk seluruh sampel, dan mengalikan dengan tanda negatif langsung menghasilkan bentuk BCE. Turunan terhadap probabilitas prediksi adalah

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial \hat{y}_i} = -\frac{1}{N} \left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right). \tag{14}$$

Karena ekspresi ini bersifat singular pada $\hat{y}_i = 0$ atau $\hat{y}_i = 1$, implementasi memakai `clamp` agar argumen log tetap berada pada interval yang aman. Jadi, secara statistik BCE memang cocok untuk keluaran probabilistik biner, tetapi secara numerik perhitungannya tetap harus dijaga dengan hati-hati.

Implementasi `BCELoss`

```
class BCELoss(_Loss):
    def __init__(self, eps: float = 1e-12) -> None:
        self.eps = eps

    def forward(self, pred, target) -> Tensor:
        p = _ensure_tensor(pred).clamp(self.eps, 1.0 - self.eps)
        t = _ensure_tensor(target)
        one = Tensor(1.0)
        return -(t * p.log() + (one - t) * (one - p).log()).mean()
```

Baris `clamp(self.eps, 1.0 - self.eps)` mencegah operasi log terhadap nol. Detail ini terlihat kecil, tetapi sangat penting untuk menjaga stabilitas ketika probabilitas prediksi berada sangat dekat dengan 0 atau 1.

Backward lokal yang dipakai `BCELoss`

```
def clamp(self, min_value, max_value):
    out = Tensor(np.clip(self.data, min_value, max_value), ...)

    def _backward():
        if self.requires_grad and self.grad is not None:
            mask = (self.data >= min_value) & (self.data <= max_value)
            self.grad += out.grad * mask

def log(self):
    out = Tensor(np.log(self.data), ...)

    def _backward():
        if self.requires_grad and self.grad is not None:
            self.grad += out.grad / self.data
```

Gradien BCE mengalir melalui operasi `log` dan operasi linear di graph komputasi. `clamp` memberi mask gradien pada interval aman sehingga titik di luar rentang tidak mendorong pembaruan numerik yang tidak valid.

Categorical Cross-Entropy untuk distribusi multikelas. Pada target one-hot $\mathbf{t}_i \in \{0,1\}^C$ dan probabilitas kelas $\mathbf{p}_i$, Categorical Cross-Entropy dihitung sebagai

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} t_{ic}\log p_{ic}. \tag{15}$$

Loss ini merupakan negative log-likelihood dari distribusi kategorikal. Karena hanya satu komponen t_{ic} bernilai 1 untuk tiap sampel, maka

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N}\sum_{i=1}^{N}\log p_{i,y_i}, \tag{16}$$

yaitu penalti langsung terhadap probabilitas yang diberikan model pada kelas benar. Kasus biner dapat dilihat sebagai kasus khusus CCE dengan $C = 2$ sehingga BCE pada dasarnya adalah bentuk tereduksi dari cross-entropy kategorikal.

Implementasi `CrossEntropyLoss` **pada graph** `Tensor`

```
class CrossEntropyLoss(_Loss):
    def __init__(self, eps: float = 1e-12) -> None:
        self.eps = eps

    def forward(self, pred, target) -> Tensor:
        p = _ensure_tensor(pred).clamp(self.eps, 1.0 - self.eps)
        target_np = target.data if isinstance(target, Tensor) else np.asarray(target
    )
        if target_np.ndim == 1:
            one_hot = _as_one_hot(target_np, p.shape[1])
            t = Tensor(one_hot, requires_grad=False)
        elif target_np.ndim == 2 and target_np.shape == p.shape:
            t = Tensor(target_np, requires_grad=False)
        return -(t * p.log()).sum(axis=1).mean()
```

Kode ini menerima dua format target, yaitu indeks kelas dan one-hot. Fleksibilitas ini membuat API pelatihan tetap konsisten di beberapa skenario eksperimen yang berbeda.

Backward lokal tambahan pada `CrossEntropyLoss`

```
def sum(self, axis=None, keepdims=False):
    out = Tensor(np.sum(self.data, axis=axis, keepdims=keepdims), ...)
```

```
    def _backward():
        if not self.requires_grad or self.grad is None:
            return
        if axis is None:
            grad = np.ones_like(self.data) * out.grad
        else:
            axes = axis if isinstance(axis, tuple) else (axis,)
            expanded = out.grad
            if not keepdims:
                for ax in sorted(axes):
                    expanded = np.expand_dims(expanded, ax)
            grad = np.ones_like(self.data) * expanded
        self.grad += grad
```

Selain `clamp` dan `log` seperti pada BCE, CCE juga memakai `sum(axis=1)` untuk mengakumulasi kontribusi dari semua kelas. Pada tahap *backward*, operasi `sum` menyiarkan gradien kembali ke tiap komponen kelas sebelum akhirnya dirata-ratakan oleh `mean`.

Implementasi `cross_entropy` stabil numerik dari logits

```
shifted = logits - np.max(logits, axis=1, keepdims=True)
logsumexp = np.log(np.sum(np.exp(shifted), axis=1, keepdims=True))
log_probs = shifted - logsumexp

if target.ndim == 1:
    losses = -log_probs[np.arange(n), indices]
elif target.ndim == 2 and target.shape == logits.shape:
    losses = -np.sum(target * log_probs, axis=1)
```

Operasi `shifted` merealisasikan trik *log-sum-exp* sehingga perhitungan log-probabilitas tetap stabil ketika nilai logit besar. Struktur ini dipakai pada lapisan fungsional NumPy untuk validasi numerik di luar graph autodiff.

Mengapa cross-entropy selaras dengan sigmoid dan softmax. Jika logits biner ditulis z_i dan probabilitas diberikan oleh $\hat{y}_i = \sigma(z_i)$, maka untuk satu sampel

$$\ell(z_i, y_i) = -y_i \log \sigma(z_i) - (1 - y_i) \log(1 - \sigma(z_i)). \tag{17}$$

Dengan menggunakan

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{18}$$

diperoleh turunan terhadap logit

$$\frac{\partial \ell}{\partial z_i} = \sigma(z_i) - y_i. \tag{19}$$

Hasil ini penting karena gradien terhadap logit menjadi sederhana dan tidak lagi memuat pembagian oleh $\hat{y}_i(1 - \hat{y}_i)$. Untuk softmax multikelas dengan logits $\mathbf{z}_i$ dan probabilitas $\mathbf{p}_i = \mathrm{softmax}(\mathbf{z}_i)$, turunan serupa menghasilkan

$$\frac{\partial \ell}{\partial z_{ic}} = p_{ic} - t_{ic}. \tag{20}$$

Identitas ini menjelaskan mengapa pasangan *sigmoid* + *BCE* dan *softmax* + *cross-entropy* sangat sering dipakai: keduanya menghasilkan gradien keluaran yang sederhana, stabil, dan langsung sebanding dengan selisih antara probabilitas prediksi dan target.

BCEWithLogitsLoss dan stabilitas numerik. Alih-alih menghitung sigmoid terlebih dahulu lalu menerapkan BCE, implementasi ini menggunakan logit z secara langsung:

$$\mathcal{L}(z, y) = \max(z, 0) - zy + \log\left(1 + e^{-|z|}\right). \tag{21}$$

Bentuk ini ekuivalen secara aljabar dengan BCE pada probabilitas sigmoid, tetapi lebih stabil secara numerik [22]. Jika $z \geq 0$, maka $|z| = z$ dan

$$\mathcal{L}(z, y) = z - zy + \log(1 + e^{-z}). \tag{22}$$

Karena

$$\log \sigma(z) = -\log(1 + e^{-z}), \qquad \log(1 - \sigma(z)) = -z - \log(1 + e^{-z}), \tag{23}$$

bentuk di atas identik dengan BCE standar. Jika $z < 0$, maka $|z| = -z$ dan

$$\mathcal{L}(z, y) = -zy + \log(1 + e^{z}), \tag{24}$$

yang sekali lagi identik dengan ekspresi BCE setelah substitusi sigmoid. Keuntungan utamanya adalah kita tidak perlu menghitung e^{z} untuk $z \gg 0$ atau e^{-z} untuk $z \ll 0$ secara tidak terkendali sehingga risiko *overflow* dan *underflow* berkurang drastis.

Implementasi `BCEWithLogitsLoss` **pada graph** `Tensor`

```
class BCEWithLogitsLoss(_Loss):
    def forward(self, pred, target) -> Tensor:
        p = _ensure_tensor(pred)
        t = _ensure_tensor(target)
        abs_p = p.relu() + (-p).relu()
        return (p.relu() - p * t + (Tensor(1.0) + (-abs_p).exp()).log()).mean()
```

Baris `abs_p = relu(p) + relu(-p)` membentuk nilai absolut tanpa memerlukan operator khusus sehingga tetap kompatibel dengan graph autodiff internal. Bentuk ini setara dengan rumus analitik di atas dan karena itu dipakai sebagai pilihan default untuk eksperimen biner pada laporan ini.

Backward lokal yang dipakai `BCEWithLogitsLoss`

```
def relu(self):
    out = Tensor(np.maximum(0.0, self.data), ...)

    def _backward():
        if self.requires_grad and self.grad is not None:
            self.grad += out.grad * (self.data > 0.0)

def exp(self):
    out = Tensor(np.exp(self.data), ...)

    def _backward():
        if self.requires_grad and self.grad is not None:
            self.grad += out.grad * out.data

def log(self):
    out = Tensor(np.log(self.data), ...)

    def _backward():
        if self.requires_grad and self.grad is not None:
            self.grad += out.grad / self.data
```

Rangkaian `relu`, `exp`, dan `log` inilah yang membentuk formulasi berbasis logit yang stabil secara numerik. Graph tetap sederhana karena setiap node hanya perlu mengetahui turunan lokalnya sendiri, sementara engine `Tensor.backward()` yang menangani penerapan *chain rule* secara global.

Implikasi pemilihan loss. Secara konseptual, MSE lebih cocok untuk regresi atau keluaran kontinu, BCE lebih cocok untuk klasifikasi biner probabilistik, CCE lebih cocok untuk klasifikasi multikelas, dan `BCEWithLogitsLoss` paling tepat dipilih ketika model mengeluarkan logit secara langsung. Dalam konteks tugas ini, pemilihan `BCEWithLogitsLoss` juga selaras dengan metrik akurasi, tetapi keduanya tetap punya peran yang berbeda. *Loss* melatih model agar menghasilkan logit yang konsisten dengan *likelihood* Bernoulli, sedangkan akurasi hanya memeriksa keputusan diskret setelah probabilitas $\sigma(z)$ ditreshold pada 0.5. Karena threshold 0.5 ekuivalen dengan syarat $z \geq 0$, objective ini secara tidak langsung mendorong logit ke sisi keputusan yang benar, sementara akurasi hanya memeriksa apakah keputusan akhirnya sudah sesuai. Jadi, `BCEWithLogitsLoss` dipilih bukan karena ia mengoptimalkan akurasi secara langsung, melainkan karena ia memberi objective yang sesuai untuk klasifikasi biner. Setelah objective ditetapkan, langkah berikutnya adalah melihat bagaimana gradien dihitung dan dipakai untuk menurunkan *loss*.

3.9 Backpropagation dan Penurunan Loss

Bagian ini membahas bagaimana gradien parameter dihitung secara efisien dan mengapa gradien tersebut dapat menurunkan *loss* secara lokal. Untuk satu sampel atau satu *mini-batch*, FFNN ditulis sebagai

$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{z}^{(l)} = \mathbf{a}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}), \quad l = 1, \ldots, L, \tag{25}$$

dengan *loss* skalar

$$\mathcal{L}(\theta) = \ell(\mathbf{a}^{(L)}, \mathbf{y}), \tag{26}$$

di mana θ menghimpun seluruh bobot dan bias. Backpropagation memperkenalkan *error signal*

$$\delta^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}, \tag{27}$$

sehingga chain rule dapat dinyatakan sebagai rekurensi antar layer [13, 12]. Pada layer keluaran berlaku

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \odot \phi^{(L)\prime}(\mathbf{z}^{(L)}), \tag{28}$$

dan untuk setiap hidden layer $l = L-1, \ldots, 1$ berlaku

$$\delta^{(l)} = \left(\delta^{(l+1)}(\mathbf{W}^{(l+1)})^\top\right) \odot \phi^{(l)\prime}(\mathbf{z}^{(l)}). \tag{29}$$

Dengan demikian, error pada suatu layer diperoleh dengan "memantulkan" error dari layer sesudahnya melalui bobot transpos, lalu mengalikannya dengan turunan lokal aktivasi. Dari sudut pandang ini, backpropagation pada dasarnya hanyalah penerapan *chain rule* pada komposisi affine-nonlinear yang berlapis.

Setelah $\delta^{(l)}$ diketahui, gradien parameter layer ke-l diperoleh langsung dari kalkulus matriks:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = (\mathbf{a}^{(l-1)})^\top \delta^{(l)}, \tag{30}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \mathbf{1}^\top \delta^{(l)}, \tag{31}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l-1)}} = \delta^{(l)}(\mathbf{W}^{(l)})^\top. \tag{32}$$

Untuk *mini-batch*, setiap baris pada $\delta^{(l)}$ merepresentasikan *error signal* dari satu sampel. Karena itu, perkalian $(\mathbf{a}^{(l-1)})^\top \delta^{(l)}$ secara simultan mengagregasi kontribusi gradien dari seluruh sampel pada batch. Inilah dasar matematis yang membuat implementasi `Linear` bisa ditulis secara tervektorisasi.

Langkah backpropagation pada loop pelatihan

```
for xb, yb in train_loader:
    logits = model(xb)
    loss = criterion(logits, yb)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Urutan ini merepresentasikan satu langkah optimisasi standar. Pemanggilan `loss.backward()` menghitung gradien seluruh parameter aktif pada batch saat ini, sedangkan `optimizer.step()` memperbarui parameter menggunakan gradien tersebut.

Aturan gradien affine pada operasi `@`

```
def __matmul__(self, other):
    out = Tensor(self.data @ other_t.data, _children=(self, other_t), _op="@")

    def _backward():
        if out.grad is None:
            return
```

```
        if self.requires_grad and self.grad is not None:
            self.grad += out.grad @ other_t.data.T
        if other_t.requires_grad and other_t.grad is not None:
            other_t.grad += self.data.T @ out.grad

    out._backward = _backward
    return out
```

Potongan ini adalah bentuk implementasi langsung dari turunan matriks pada persamaan sebelumnya. Suku `out.grad @ other_t.data.T` merealisasikan $\partial\mathcal{L}/\partial X$, sedangkan `self.data.T @ out.grad` merealisasikan $\partial\mathcal{L}/\partial W$.

Kasus khusus keluaran biner dengan `BCEWithLogitsLoss`. Untuk satu neuron keluaran dengan logit batch $Z = XW + b$ dan target biner $Y \in \{0,1\}^{B\times 1}$, loss rata-rata `BCEWithLogitsLoss` memberi error signal keluaran

$$\delta^{(L)} = \frac{1}{B}\left(\sigma(Z) - Y\right). \tag{33}$$

Akibatnya gradien parameter pada layer terakhir mengambil bentuk eksplisit

$$\nabla_W \mathcal{L} = X^\top \delta^{(L)}, \tag{34}$$

$$\nabla_b \mathcal{L} = \mathbf{1}^\top \delta^{(L)}, \tag{35}$$

$$\nabla_X \mathcal{L} = \delta^{(L)} W^\top. \tag{36}$$

Contoh ini menunjukkan bahwa kombinasi `sigmoid` implisit dan *cross-entropy* mereduksi gradien keluaran menjadi selisih antara probabilitas prediksi dan target, sedangkan hidden layer tetap mengikuti rekurensi umum yang sudah diturunkan sebelumnya.

Keterkaitannya dengan implementasi bersifat langsung. Kelas `Linear` membangun node perkalian matriks dan penjumlahan bias yang mewakili turunan affine. Kelas-kelas aktivasi dan *loss* menyediakan turunan lokal masing-masing sehingga suku $\phi'(\mathbf{z}^{(l)})$ dan $\partial\ell/\partial\mathbf{a}^{(L)}$ tersedia di graph. Saat `loss.backward()` dipanggil pada `Tensor`, gradien awal 1 ditanamkan pada node *loss* skalar, lalu propagasi terbalik dilakukan sehingga setiap `_backward` mengakumulasi kontribusi yang ekuivalen dengan rekurensi $\delta^{(l)}$ di atas. Setelah gradien tersimpan pada `weight.grad` dan `bias.grad`, `optimizer.step()` menerapkan langkah penurunan sesuai aturan SGD atau Adam pada Subbagian 3.11. Jadi, *autodiff core* di sini bukan menggantikan teori backpropagation, tetapi mewujudkannya secara generik di atas graph komputasi.

3.10 Mekanisme Automatic Differentiation

Automatic differentiation pada sistem ini menggunakan reverse-mode AD pada graph komputasi berarah tanpa siklus $G = (V, E)$. Setiap operasi diferensiabel yang menghasilkan `Tensor` baru akan membentuk node baru yang menyimpan tiga hal: nilai maju, himpunan parent $\mathrm{pa}(v)$ melalui `_prev`, dan operator gradien lokal melalui `_backward`. Karena graph dibangun mengikuti urutan eksekusi program yang benar-benar terjadi, mekanisme ini bersifat *define-by-run* dan sejalan dengan formulasi autodiff modern berbasis graph komputasi dinamis [14, 20, 26].

Secara matematis, reverse-mode AD menghitung *vector-Jacobian product* dari output skalar ke seluruh node antara. Untuk *loss* skalar $\mathcal{L}$ dan node antara u, gradien global memenuhi

$$\bar{u} \equiv \frac{\partial\mathcal{L}}{\partial u} = \sum_{v\in\mathrm{ch}(u)} \bar{v}\,\frac{\partial v}{\partial u}, \tag{37}$$

dengan $\mathrm{ch}(u)$ menyatakan himpunan anak node u. Jadi, reverse-mode AD tidak memperkenalkan rumus baru di luar backpropagation; ia hanya mengeksekusi *chain rule* pada graph komputasi yang lebih umum melalui akumulasi gradien lokal pada setiap edge.

Secara algoritmik, `backward()` terdiri atas dua tahap. Tahap pertama membangun urutan topologis dengan DFS mulai dari node *loss*. Tahap kedua menelusuri urutan tersebut secara terbalik dan memanggil `_backward` pada setiap node. Kompleksitas waktunya linear terhadap ukuran graph yang aktif pada satu *mini-batch*, yaitu $\mathcal{O}(|V| + |E|)$.

Implementasi inti reverse-mode pada `Tensor.backward`

```
def backward(self, grad=None):
    if grad is None:
        grad = np.ones_like(self.data)

    self.grad = self.grad + grad if self.grad is not None else grad
    topo, visited = [], set()

    def build(node):
        if id(node) in visited:
            return
        visited.add(id(node))
        for child in node._prev:
            build(child)
        topo.append(node)

    build(self)
    for node in reversed(topo):
        node._backward()
```

Fungsi ini memperlihatkan alur *define-by-run* secara langsung. Graph dibentuk saat *forward*, lalu urutan topologis dibangun hanya untuk node yang benar-benar relevan terhadap *loss* saat ini. Setelah itu, eksekusi terbalik pada `topo` menerapkan *chain rule* global melalui callback lokal pada setiap operasi.

Algoritma 1: Reverse-mode autodiff untuk `Tensor.backward`

Data: Node loss skalar `loss`, gradien awal opsional `grad`
Result: Gradien terakumulasi pada semua parent yang relevan
if $loss.requires_grad = False$ **then**
  return;
else
  if $grad = \varnothing$ **then**
    set `grad` $\leftarrow$ **1** dengan shape `loss` ; /* seed grad */
  end
  `loss.grad` $\leftarrow$ `loss.grad` + `grad`;
  build topological order dengan DFS dari node `loss`;
  foreach *node dalam urutan topologis terbalik* **do**
    panggil `node._backward()`;
  end
end

Dalam notasi multivariabel, gradien parameter tetap mengikuti

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_k \frac{\partial \mathcal{L}}{\partial u_k} \frac{\partial u_k}{\partial \theta}. \tag{38}$$

Salah satu detail implementasi yang penting adalah penanganan broadcasting. Pada *forward pass*, bias atau skalar sering diekspansi secara implisit oleh NumPy sehingga shape gradien hasil operasi tidak selalu sama dengan shape parameter asal. Fungsi `_unbroadcast` mengatasi hal ini dengan menjumlahkan gradien pada dimensi ekstra atau dimensi bernilai satu sehingga gradien akhirnya kembali kompatibel dengan parameter yang sedang dioptimisasi. Mekanisme ini krusial agar pembaruan bias pada operasi berbasis batch tetap sah secara matematis.

Contoh gradien lokal operator dan koreksi shape gradien

```
def _unbroadcast(grad, target_shape):
    g = grad
    while len(g.shape) > len(target_shape):
        g = np.sum(g, axis=0)
    for axis, (g_dim, t_dim) in enumerate(zip(g.shape, target_shape)):
        if t_dim == 1 and g_dim != 1:
            g = np.sum(g, axis=axis, keepdims=True)
```

```
    return g.reshape(target_shape)

def _backward_matmul(self, other_t, out):
    if out.grad is None:
        return
    if self.requires_grad and self.grad is not None:
        self.grad += out.grad @ other_t.data.T
    if other_t.requires_grad and other_t.grad is not None:
        other_t.grad += self.data.T @ out.grad
```

Potongan ini menunjukkan dua detail penting. Pertama, aturan gradien lokal untuk perkalian matriks ditulis eksplisit melalui operasi transpose. Kedua, `_unbroadcast` memastikan gradien kembali ke shape parameter asal ketika *forward pass* memakai broadcasting NumPy.

3.11 Update Bobot, Optimizer, dan Regularisasi

Pembahasan optimizer berikut sengaja diikat langsung ke implementasi agar alur antara teori dan kode tetap konsisten. Pada iterasi ke-t dengan *mini-batch* B_t, objective yang dioptimisasi adalah

$$J_t(\theta) = \frac{1}{|B_t|} \sum_{(x_i, y_i) \in B_t} \ell(f_\theta(x_i), y_i) + \lambda_1 \|\theta\|_1 + \frac{\lambda_2}{2} \|\theta\|_2^2. \tag{39}$$

Jika gradien loss batch ditulis $g_t = \nabla_\theta \mathcal{L}_{B_t}(\theta_t)$ maka gradien efektif yang dipakai update menjadi [17, 18]

$$\tilde{g}_t = g_t + \lambda_1 \operatorname{sign}(\theta_t) + \lambda_2 \, \theta_t. \tag{40}$$

Bentuk ini identik dengan ekspresi `reg_grad` yang muncul di kode.

Abstraksi dasar `Optimizer`

```
class Optimizer:
    def __init__(self, params, defaults):
        self.defaults = defaults.copy()
        self.state = {}
        self.param_groups = []
        self.add_param_group({"params": list(params)})

    @property
    def params(self):
        all_params = []
        for group in self.param_groups:
            all_params.extend(group["params"])
        return all_params

    def zero_grad(self):
        for p in self.params:
            p.zero_grad()
```

Kelas dasar ini mengelompokkan parameter ke dalam `param_groups`, lalu menyediakan `zero_grad()` untuk mereset gradien. Struktur ini membuat `SGD` dan `Adam` dapat berbagi mekanisme manajemen parameter dan penyimpanan *state*.

SGD. Untuk `SGD`, aturan update langsung ditulis

$$\theta_{t+1} = \theta_t - \eta \, \tilde{g}_t = \theta_t - \eta \left(g_t + \lambda_1 \mathrm{sign}(\theta_t) + \lambda_2 \theta_t \right). \tag{41}$$

Implementasi persisnya ada pada blok berikut.

Implementasi `SGD.step()`

```
def step(self) -> None:
    for group in self.param_groups:
        lr = float(group["lr"])
```

```
        l1_lambda = float(group["l1_lambda"])
        l2_lambda = float(group["l2_lambda"])
        for p in group["params"]:
            if p.grad is None:
                continue
            reg_grad = 0.0
            if l1_lambda != 0.0:
                reg_grad = reg_grad + l1_lambda * np.sign(p.data)
            if l2_lambda != 0.0:
                reg_grad = reg_grad + l2_lambda * p.data
            p.data -= lr * (p.grad + reg_grad)
```

Baris `p.data -= lr * (p.grad + reg_grad)` merealisasikan persamaan update di atas secara elemen demi elemen. Komponen `np.sign(p.data)` mewakili kontribusi L1, sedangkan `p.data` mewakili kontribusi L2.

Adam. Pada `Adam`, gradien efektif yang sama dipakai untuk membentuk momen:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\tilde{g}_t, \tag{42}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\tilde{g}_t^{\odot 2}, \tag{43}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{44}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{45}$$

Bias correction pada $\hat{m}_t$ dan $\hat{v}_t$ penting terutama pada fase awal *training*, ketika m_t dan v_t masih dekat nol.

Implementasi `Adam.step()`

```
def step(self) -> None:
    self.t += 1
    for group in self.param_groups:
        lr = float(group["lr"])
        beta1 = float(group["beta1"])
        beta2 = float(group["beta2"])
        eps = float(group["eps"])
        l1_lambda = float(group["l1_lambda"])
        l2_lambda = float(group["l2_lambda"])
        bias_correction1 = 1.0 - beta1**self.t
        bias_correction2 = 1.0 - beta2**self.t

        for p in group["params"]:
            if p.grad is None:
                continue
            state = self.state.setdefault(p, {})
            if "m" not in state:
                state["m"] = np.zeros_like(p.data)
                state["v"] = np.zeros_like(p.data)

            reg_grad = 0.0
            if l1_lambda != 0.0:
                reg_grad = reg_grad + l1_lambda * np.sign(p.data)
            if l2_lambda != 0.0:
                reg_grad = reg_grad + l2_lambda * p.data

            grad = p.grad + reg_grad
            state["m"] = beta1 * state["m"] + (1.0 - beta1) * grad
            state["v"] = beta2 * state["v"] + (1.0 - beta2) * (grad * grad)
            m_hat = state["m"] / bias_correction1
            v_hat = state["v"] / bias_correction2
            p.data -= lr * (m_hat / (np.sqrt(v_hat) + eps))
```

Urutan operasinya mengikuti formula secara langsung: tambahkan regularisasi ke gradien, perbarui momen, lakukan *bias correction*, lalu baru perbarui parameter. Implementasi ini memakai penalti L2 berbasis gradien sehingga perilakunya bukan *decoupled weight decay* seperti pada AdamW [2, 19].

Konsistensi antara persamaan dan kode membuat analisis eksperimen pada bagian berikut lebih mudah ditelusuri, dari level teori sampai level implementasi.

3.12 Normalisasi RMSNorm

RMSNorm pada laporan ini dibahas terutama dari sisi implementasi. Operasi intinya adalah normalisasi RMS per sampel:

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}}. \tag{46}$$

Setelah dinormalisasi, vektor dapat dikalikan dengan parameter trainable `weight` jika `affine=True`. Tujuan praktisnya adalah menjaga skala aktivasi antar layer agar lebih stabil selama *training*.

Implementasi `RMSNorm`

```
class RMSNorm(Module):
    def __init__(self, normalized_shape, eps=1e-8, affine=True):
        self.normalized_shape = int(normalized_shape)
        self.eps = float(eps)
        self.weight = (Tensor(np.ones((1, self.normalized_shape)),
                              requires_grad=True) if affine else None)

    def forward(self, x: Tensor) -> Tensor:
        if x.shape[-1] != self.normalized_shape:
            raise ValueError("shape mismatch")
        rms = ((x * x).mean(axis=-1, keepdims=True) + self.eps) ** 0.5
        out = x / rms
        if self.weight is not None:
            out = out * self.weight
        return out
```

Kode di atas menunjukkan tiga hal utama. Pertama, ada validasi dimensi agar ukuran fitur terakhir sesuai dengan `normalized_shape`. Kedua, normalisasi dihitung langsung dari operasi tensor inti agar gradien tetap mengalir otomatis melalui graph. Ketiga, parameter `weight` dibuat opsional agar eksperimen bisa membandingkan RMSNorm affine dan non-affine.

3.13 Mekanisme Data, Pelatihan, dan Persistensi

Pipeline data memisahkan preprocessing train dan validation dengan benar: `StandardScaler` di-fit pada data train lalu transformasinya diterapkan ke validation; fitur kategorikal diproses dengan `OneHotEncoder(handle_unknown="ignore")` [23, 24, 21]. Pada level implementasi, pasangan fitur-target terlebih dahulu dibungkus oleh `ArrayDataset` yang menormalkan bentuk array dan menjaga konsistensi jumlah sampel lalu diiterasikan melalui `DataLoader` dengan opsi `shuffle`, `batch_size`, `seed`, dan `drop_last`.

Preprocessing pada `load_dataset_features`

```
y_all = np.array(
    [1.0 if row[TARGET_COL] == "Placed" else 0.0 for row in rows],
    dtype=np.float64,
).reshape(-1, 1)
train_rows, val_rows, y_train, y_val = train_test_split(
    rows, y_all, test_size=0.2, random_state=split_seed, shuffle=True
)

num_idx = list(range(len(NUMERIC_COLS)))
cat_idx = list(range(len(NUMERIC_COLS), len(NUMERIC_COLS) + len(ONEHOT_COLS)))
preprocessor = ColumnTransformer(
    transformers=[
```

```
        ("num", StandardScaler(), num_idx),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_idx
    ),
    ],
    remainder="drop",
    verbose_feature_names_out=False,
)
x_train = preprocessor.fit_transform(x_train_raw).astype(np.float64)
x_val = preprocessor.transform(x_val_raw).astype(np.float64)
```

Label target dibentuk sebagai biner 0/1, lalu data dipecah menjadi train dan validation dengan seed tetap. `ColumnTransformer` memisahkan fitur numerik dan kategorikal; `StandardScaler` di-fit hanya pada train, sedangkan `OneHotEncoder(handle_unknown="ignore")` menjaga pipeline tetap aman jika validation memuat kategori yang tidak muncul pada train. Urutan `fit_transform` pada train dan `transform` pada validation penting untuk menjaga evaluasi tetap bebas dari *data leakage*.

Siklus pelatihan per *mini-batch* adalah: *forward* → *loss* → `optimizer.zero_grad()` → `loss.backward()` → `optimizer.step()`. Riwayat *train loss* dan *validation loss* disimpan per epoch untuk analisis konvergensi. Selain itu, framework juga menyediakan persistensi model (`Module.save/load`) dan persistensi optimizer (`state_dict/load_state_dict`) agar eksperimen bisa direproduksi atau dilanjutkan.

Loop pelatihan pada `train_experiment` di `src/main.py`

```
for epoch in range(epochs):
    for xb, yb in train_loader:
        logits = model(xb)
        loss = criterion(logits, yb)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss, _, _, _ = evaluate_binary(model, x_train, y_train, criterion)
    val_loss, val_acc, _, _ = evaluate_binary(model, x_val, y_val, criterion)
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    val_accs.append(val_acc)
```

Potongan ini merupakan implementasi langsung dari prosedur pelatihan *mini-batch* yang dipakai pada eksperimen. Struktur loop-nya memperlihatkan dengan jelas perbedaan antara pembaruan parameter di level batch dan evaluasi metrik di level epoch.

3.14 Analisis Distribusi Bobot dan Gradien

Untuk menganalisis geometri parameter selama eksperimen, framework menyediakan dua method pada kelas `Sequential`. Method pertama, `plot_weight_distribution()`, dipakai untuk memvisualisasikan parameter akhir. Method kedua, `plot_gradient_distribution()`, dipakai untuk memvisualisasikan gradien pada parameter yang sama.

Dua method plotting pada `Sequential`

```
def plot_weight_distribution(self, layer_indices: list[int]) -> None:
    self._plot_distribution(layer_indices, use_grad=False)

def plot_gradient_distribution(self, layer_indices: list[int]) -> None:
    self._plot_distribution(layer_indices, use_grad=True)
```

Kedua method memakai helper yang sama sehingga perbedaan visualisasi hanya berasal dari sumber datanya: `.data` untuk bobot dan `.grad` untuk gradien.

Inti perhitungan vektor layer dan histogram

```
def _plot_distribution(self, layer_indices: list[int], *, use_grad: bool) -> None:
    linear_layers = [layer for layer in self.layers if hasattr(layer, "weight")]
    for ax, idx in zip(axes, layer_indices):
        layer = linear_layers[idx]
        weight_values = layer.weight.grad if use_grad else layer.weight.data
        values = [weight_values.reshape(-1)]
        if getattr(layer, "bias", None) is not None:
            bias_values = layer.bias.grad if use_grad else layer.bias.data
            values.append(bias_values.reshape(-1))
        flat = np.concatenate(values)
        ax.hist(flat, bins=30, edgecolor="black", alpha=0.85)
```

Untuk linear layer ke-l dengan bobot $\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l-1} \times d_l}$ dan bias $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ vektor bobot yang dipetakan ke histogram didefinisikan sebagai

$$\mathbf{w}^{(l)} = \begin{bmatrix} \mathrm{vec}\big(\mathbf{W}^{(l)}\big) \\ \mathbf{b}^{(l)} \end{bmatrix} \in \mathbb{R}^{m_l}, \qquad m_l = d_{l-1} d_l + d_l. \tag{47}$$

Jika rentang nilai dibagi menjadi K bin tak tumpang tindih $\{I_k\}_{k=1}^{K}$ maka frekuensi histogram bobot pada bin ke-k adalah

$$c_{W,k}^{(l)} = \sum_{j=1}^{m_l} \mathbf{1}\Big[w_j^{(l)} \in I_k\Big], \qquad k = 1, \ldots, K. \tag{48}$$

Implementasi memakai $K = 30$ sesuai argumen `bins=30`.

Histogram gradien dihitung dengan konstruksi identik tetapi vektor bobot diganti oleh vektor gradien

$$\mathbf{g}^{(l)} = \begin{bmatrix} \mathrm{vec}(\nabla_{\mathbf{W}^{(l)}} \mathcal{L}_{\mathrm{train}}(\theta)) \\ \nabla_{\mathbf{b}^{(l)}} \mathcal{L}_{\mathrm{train}}(\theta) \end{bmatrix}, \tag{49}$$

dengan objective train

$$\mathcal{L}_{\mathrm{train}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), y_i)\,. \tag{50}$$

Frekuensi histogram gradien pada bin ke-k menjadi

$$c_{G,k}^{(l)} = \sum_{j=1}^{m_l} \mathbf{1}\Big[g_j^{(l)} \in I_k\Big]\,. \tag{51}$$

Perhitungan gradien acuan sebelum plotting

```
def attach_full_batch_gradients(model, x, y, criterion) -> None:
    for p in model.parameters():
        p.zero_grad()
    logits = model(Tensor(x))
    loss = criterion(logits, Tensor(y))
    loss.backward()
```

Gradien untuk plot tidak diaproksimasi dengan beda hingga. Gradien dihitung langsung oleh engine autodiff pada objective full-batch sehingga nilai pada histogram merupakan turunan eksak dari graph komputasi model terhadap parameter pada titik evaluasi tersebut.

Sampai titik ini, laporan telah mengaitkan struktur kelas, formulasi matematis, dan alur pelatihan ke dalam satu kerangka yang utuh. Bagian berikutnya memakai kerangka tersebut untuk mengevaluasi perilaku empiris sistem secara terkontrol.

4 Hasil Pengujian

Bagian ini tidak hanya berisi angka hasil akhir, tetapi juga mencoba melihat apakah keputusan desain pada bagian implementasi benar-benar berpengaruh ke perilaku model. Karena itu, setiap eksperimen

dibuat sebagai studi ablation yang terkontrol. Ide dasarnya sederhana: pada setiap studi, hanya satu faktor yang diubah sementara komponen lain dijaga tetap. Dengan cara ini, perubahan metrik bisa lebih masuk akal untuk dikaitkan dengan faktor yang sedang diuji.

Dataset dipisahkan secara *stratified hold-out* menjadi 80% data latih dan 20% data validasi agar proporsi kelas tetap terjaga pada kedua split. Detail preprocessing sudah dijelaskan pada Subbagian 3.13. Saat evaluasi, operator transformasi hanya di-fit pada data latih lalu diterapkan ke data validasi agar tidak terjadi *data leakage*. Setelah preprocessing, diperoleh 8000 sampel latih, 2000 sampel validasi, dan 28 fitur masukan efektif.

Konfigurasi acuan eksperimen adalah FFNN klasifikasi biner dengan tiga hidden layer berukuran 32, urutan aktivasi GELU–SELU–GELU, inisialisasi He, optimizer SGD dengan *learning rate* 0.03, regularisasi L2 sebesar 10^{-4}, batch size 256, horizon pelatihan 160 epoch, serta seed tetap 123 untuk inisialisasi dan pengacakan *mini-batch*. *Loss* utama yang digunakan adalah `BCEWithLogitsLoss`. Pemilihan *loss* ini juga cukup natural karena targetnya biner dan model menghasilkan logit skalar. Jadi, objective yang dipakai memang cocok dengan bentuk keluaran model dan tetap aman secara numerik.

Secara formal, untuk parameter θ, data validasi $\{(x_i, y_i)\}_{i=1}^{n_{\text{val}}}$, dan probabilitas prediksi $p_\theta(x_i)$, dua metrik utama yang dicatat adalah

$$\hat{y}_i(\theta) = \mathbb{I}[p_\theta(x_i) \geq 0.5]\,, \tag{52}$$

$$\text{Acc}(\theta) = \frac{1}{n_{\text{val}}} \sum_{i=1}^{n_{\text{val}}} \mathbb{I}[\hat{y}_i(\theta) = y_i]\,, \tag{53}$$

serta

$$\mathcal{L}_{\text{val}}(\theta) = -\frac{1}{n_{\text{val}}} \sum_{i=1}^{n_{\text{val}}} \left[y_i \log p_\theta(x_i) + (1 - y_i) \log\left(1 - p_\theta(x_i)\right)\right]. \tag{54}$$

Akurasi dipilih untuk mengukur kualitas keputusan akhir, sedangkan *validation loss* dipilih untuk melihat kualitas probabilitas yang dihasilkan model. Dalam setting `BCEWithLogitsLoss`, keduanya memang saling berhubungan. Karena prediksi kelas 1 diberikan oleh $\sigma(z) \geq 0.5$, keputusan klasifikasi setara dengan syarat logit $z \geq 0$. Artinya, *loss* mendorong model agar logit bergerak ke sisi yang benar, sedangkan akurasi hanya memeriksa apakah hasil akhirnya sudah benar atau belum. Karena itu, dua metrik ini tetap perlu dilaporkan bersama. Model dengan *loss* yang lebih baik belum tentu langsung punya akurasi thresholded yang lebih tinggi.

Selain kurva *loss* per epoch dan metrik akhir, beberapa eksperimen juga melihat distribusi bobot dan gradien pada akhir pelatihan. Ini penting karena dua konfigurasi bisa saja punya metrik validasi yang mirip, tetapi perilaku optimisasinya berbeda. Distribusi bobot dipakai untuk melihat apakah parameter cenderung menyusut atau tidak, sedangkan distribusi gradien dipakai untuk melihat apakah update yang dihasilkan cenderung stabil atau justru menyebar terlalu lebar.

Rentang hiperparameter yang diuji juga dipilih supaya cukup beragam tetapi masih relevan untuk data tabular berukuran sedang. Width diuji pada $\{8, 16, 24, 32\}$ untuk mewakili kapasitas kecil sampai menengah. Depth diuji pada $\{2, 3, 4\}$ untuk melihat dampak penambahan layer tanpa membuat jaringan terlalu dalam. *Learning rate* diuji pada $\{0.1, 0.03, 0.005\}$ untuk mewakili pengaturan agresif, menengah, dan konservatif. Inisialisasi membandingkan `zero`, Xavier, dan He. `Zero` dipakai sebagai kontrol negatif sedangkan Xavier dan He dipakai sebagai baseline utama. Regularisasi diuji pada tingkat lemah ($L1 = 10^{-5}$, $L2 = 10^{-4}$) agar efeknya masih bisa dibaca sebagai koreksi ringan. RMSNorm diuji sebagai saklar biner pada arsitektur yang sama agar pengaruhnya lebih mudah dilihat. Terakhir, komparasi dengan `sklearn MLPClassifier` dibuat dengan setup yang semirip mungkin supaya perbandingannya tetap adil.

4.1 Skema Preprocessing

Pipeline preprocessing mengikuti struktur semantik fitur pada dataset. Variabel target `placement_status` dipetakan menjadi label biner

$$y = \begin{cases} 1, & \text{jika status adalah Placed,} \\ 0, & \text{jika selain itu.} \end{cases} \tag{55}$$

Pemetaan ini konsisten dengan objective klasifikasi biner yang digunakan model dan memungkinkan keluaran berupa satu logit yang kemudian dapat ditafsirkan sebagai probabilitas penempatan kerja setelah transformasi sigmoid.

Fitur masukan kemudian dipisahkan menjadi dua kelompok. Kelompok numerik terdiri atas `cgpa`, `backlogs`, `internship_count`, `aptitude_score`, `communication_score`, dan `internship_quality_score`. Kelompok kategorikal terdiri atas `college_tier`, `country`, `university_ranking_band`, `specialization`, dan `industry`. Pemisahan ini diperlukan karena dua tipe fitur tersebut memang membutuhkan perlakuan yang berbeda: fitur numerik membawa skala metrik yang bermakna, sedangkan fitur kategorikal hanya membawa identitas diskret tanpa urutan linear yang aman untuk diasumsikan.

Untuk fitur numerik, digunakan standardisasi affine

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \tag{56}$$

dengan μ_j dan σ_j dihitung hanya dari data latih. Transformasi ini dipilih karena FFNN dilatih dengan optimisasi berbasis gradien. Jika skala fitur sangat berbeda, kondisi objective bisa memburuk dan konvergensi menjadi lebih lambat. Standardisasi membantu menyamakan orde magnitudo antar fitur numerik, mengurangi dominasi fitur yang skalanya besar, dan membuat satu nilai *learning rate* lebih masuk akal untuk dipakai secara global.

Untuk fitur kategorikal, digunakan one-hot encoding. Jika suatu fitur kategorikal memiliki himpunan kategori $\mathcal{C} = \{c_1, \ldots, c_K\}$, maka suatu observasi dengan kategori c_k dipetakan ke vektor basis

$$\mathbf{e}_k = (0, \ldots, 0, 1, 0, \ldots, 0) \in \{0, 1\}^K. \tag{57}$$

Pilihan ini sengaja diambil agar model tidak mengasumsikan adanya hubungan ordinal palsu antar kategori. Opsi `handle_unknown="ignore"` digunakan agar jika pada data validasi muncul kategori yang belum pernah terlihat saat pelatihan, pipeline tetap terdefinisi dengan memetakan komponen terkait ke nol, alih-alih gagal saat inferensi.

Secara keseluruhan, preprocessing dapat dipandang sebagai pemetaan

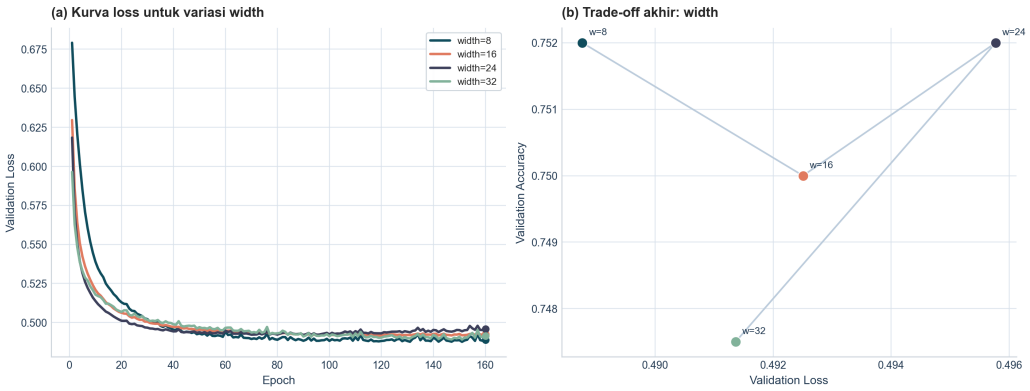
$$T(x) = \big[T_{\mathrm{num}}(x_{\mathrm{num}})\,;\, T_{\mathrm{cat}}(x_{\mathrm{cat}})\big], \tag{58}$$

dengan T_{num} adalah standardisasi per fitur dan T_{cat} adalah one-hot encoding. Operator gabungan ini di-fit pada data latih lalu diaplikasikan tanpa *refit* ke data validasi. Prinsip ini krusial agar evaluasi benar-benar mencerminkan kemampuan generalisasi ke data yang tidak terlihat, bukan kemampuan model memanfaatkan statistik dari populasi validasi.

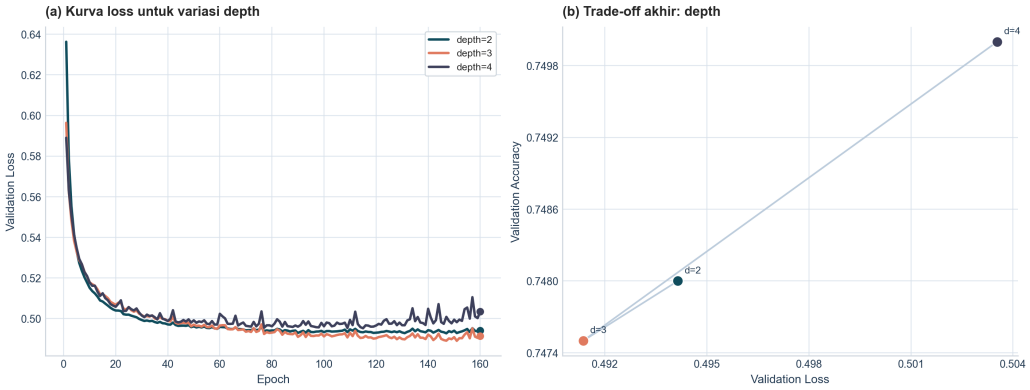
Setelah preprocessing, seluruh fitur berada dalam representasi numerik `float64` yang kompatibel dengan `Tensor`. Dimensi akhir sebesar 28 muncul dari penjumlahan 6 fitur numerik terstandarisasi dan fitur-fitur dummy hasil ekspansi kategorikal. Dengan kata lain, preprocessing di sini bukan sekadar tahap pembersihan data. Tahap ini adalah transformasi representasi yang menjembatani data tabular yang heterogen menjadi vektor kontinu yang dapat diproses secara sah oleh layer `Linear`.

4.2 Pengaruh Depth dan Width

Pada bagian ini, kapasitas model diubah lewat dua arah: lebar hidden layer dan jumlah hidden layer. Pada eksperimen width, depth dijaga tetap 3 agar yang berubah hanya ukuran representasi di tiap layer. Pada eksperimen depth, width dijaga tetap 32 agar yang berubah hanya banyaknya layer. Pemisahan ini penting karena depth dan width sama-sama menambah kapasitas model tetapi efeknya tidak sama. Width menambah banyaknya fitur laten yang bisa dipelajari, sedangkan depth menambah kemampuan model untuk membentuk komposisi nonlinier yang lebih kompleks. Hasil eksperimen menunjukkan bahwa kapasitas yang lebih besar tidak otomatis membuat generalisasi lebih baik. Pada variasi width dengan depth tetap 3, width=8 dan width=24 sama-sama memberi akurasi tertinggi 0.7520, tetapi width=8 memberi *validation loss* terendah. Pada variasi depth dengan width tetap 32, depth=4 memberi akurasi tertinggi 0.7500, tetapi depth=3 memberi *validation loss* paling rendah. Pola ini menunjukkan bahwa model yang paling akurat belum tentu menjadi model dengan loss terbaik.



Gambar 1: Eksperimen width: panel kiri menunjukkan kurva validation loss sepanjang epoch, sedangkan panel kanan merangkum trade-off validation loss dan validation accuracy pada titik akhir pelatihan.



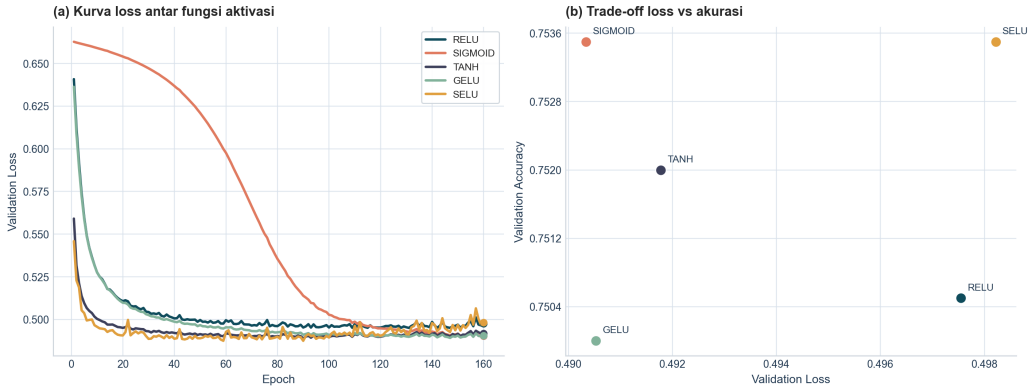
Gambar 2: Eksperimen depth: panel kiri menunjukkan dinamika validation loss, sedangkan panel kanan menampilkan trade-off akhir antara loss dan akurasi untuk tiap kedalaman jaringan.

4.3 Pengaruh Fungsi Aktivasi

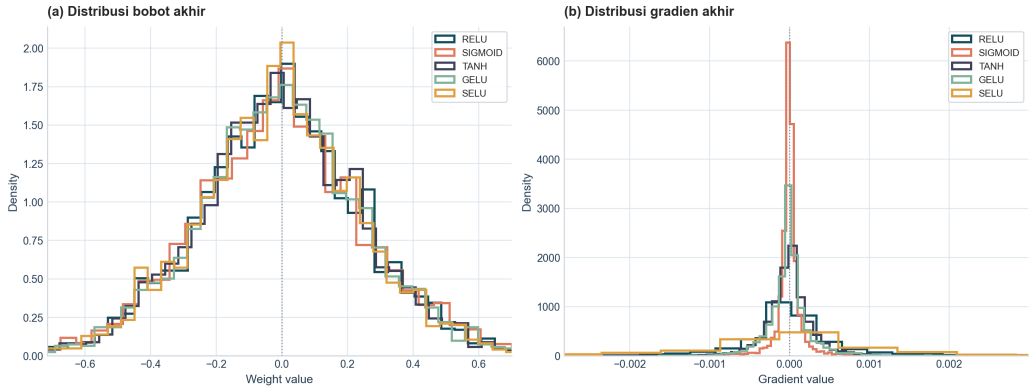
Pada studi ini, arsitektur, optimizer, *learning rate*, regularisasi, dan inisialisasi dibuat sama. Satu-satunya bagian yang diubah adalah fungsi aktivasi. Dengan cara ini, perbedaan hasil yang muncul lebih mudah dikaitkan ke pilihan aktivasi. Secara umum, aktivasi yang gradiennya lebih stabil biasanya membantu proses training, sedangkan aktivasi yang terlalu mudah jenuh bisa membuat optimisasi melambat. Hasil pada Gambar 3 dan Gambar 4 menunjukkan bahwa pilihan aktivasi memang memengaruhi kecepatan konvergensi dan kualitas hasil akhir. Pada run terbaru, `sigmoid` dan `selu` sama-sama memberi akurasi tertinggi 0.7535, sedangkan `sigmoid` juga memberi *validation loss* terendah 0.490337. Ini menunjukkan bahwa hasil aktivasi bisa cukup dekat satu sama lain dan perubahan kecil pada setup eksperimen bisa mengubah aktivasi mana yang terlihat paling unggul.

4.4 Pengaruh *Learning Rate*

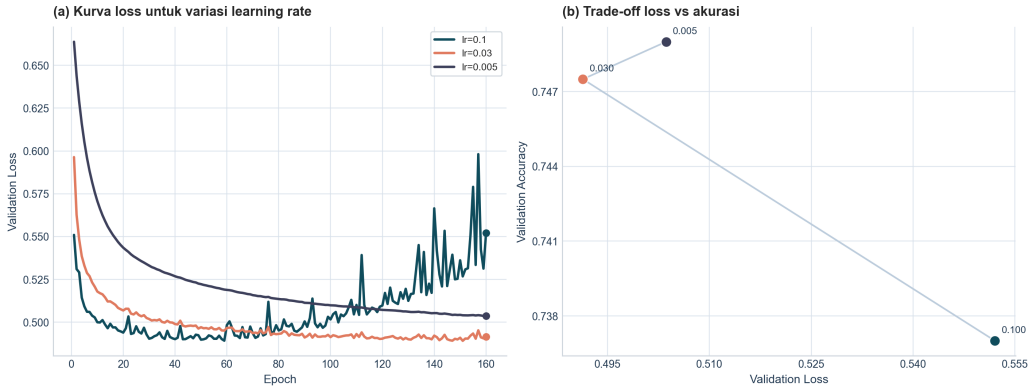
Learning rate menentukan seberapa besar langkah update yang diambil model saat training. Karena itu, eksperimen ini memakai tiga nilai yang cukup berjauhan: 0.10, 0.03, dan 0.005. Tujuannya supaya lebih jelas melihat perbedaan antara kondisi yang terlalu agresif, cukup pas, dan terlalu hati-hati. Hasil pada Gambar 5 menunjukkan bahwa *learning rate* besar seperti 0.10 membuat *validation loss* lebih tidak stabil. Pada run terbaru, lr=0.03 memberi *validation loss* terbaik 0.491365 sedangkan lr=0.005 memberi akurasi sedikit lebih tinggi, yaitu 0.7490. Dari sini terlihat bahwa pemilihan *learning rate* sangat berpengaruh ke kestabilan training dan hasil akhir.



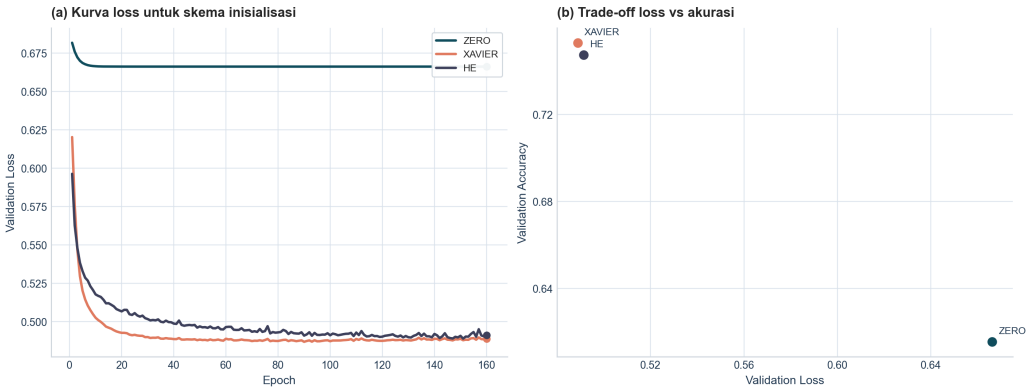
Gambar 3: Eksperimen fungsi aktivasi: panel kiri menunjukkan kurva validation loss, sedangkan panel kanan memetakan trade-off validation loss dan validation accuracy untuk setiap aktivasi.



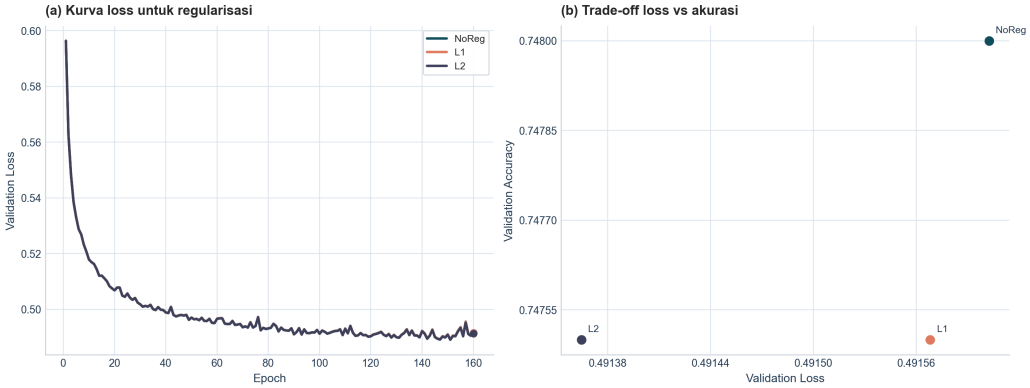
Gambar 4: Eksperimen fungsi aktivasi: panel kiri memperlihatkan distribusi bobot akhir, sedangkan panel kanan memperlihatkan distribusi gradien akhir untuk tiap aktivasi.



Gambar 5: Eksperimen *learning rate*: panel kiri menunjukkan kurva validation loss, sedangkan panel kanan merangkum trade-off antara validation loss dan validation accuracy pada akhir pelatihan.



Gambar 6: Eksperimen inisialisasi bobot: panel kiri menunjukkan kurva validation loss untuk setiap skema yang diuji, sedangkan panel kanan membandingkan trade-off akhir antara loss dan akurasi.



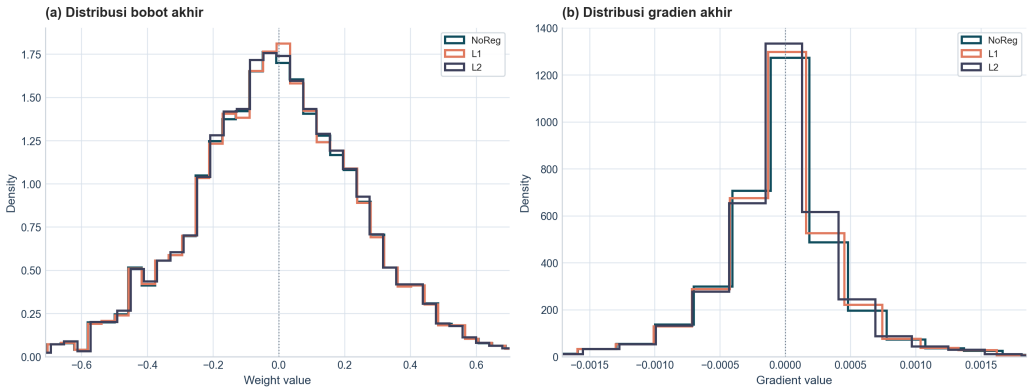
Gambar 7: Eksperimen regularisasi: panel kiri menunjukkan kurva validation loss untuk NoReg, L1, dan L2, sedangkan panel kanan merangkum trade-off akhir antara validation loss dan validation accuracy.

4.5 Pengaruh Inisialisasi Bobot

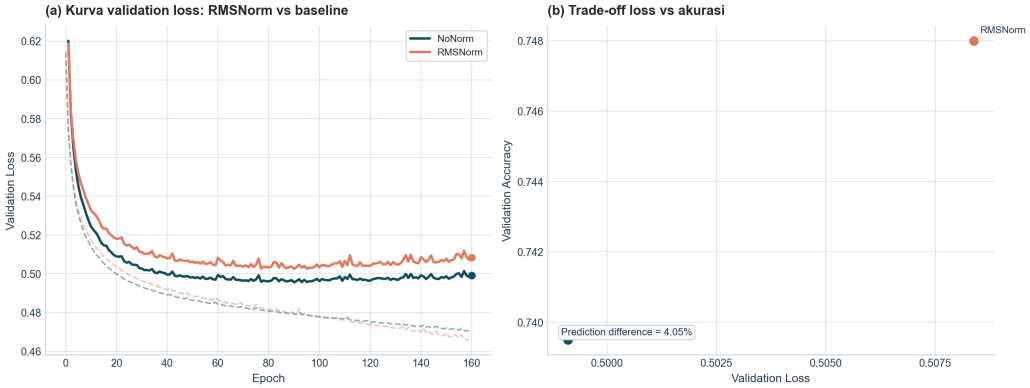
Eksperimen ini membandingkan tiga skema inisialisasi, yaitu `zero`, Xavier, dan He [4, 5]. Secara intuitif, bagian ini ingin melihat apakah pilihan nilai awal bobot benar-benar memengaruhi proses training. `Zero` dipakai sebagai kontrol negatif karena skema ini tidak memecah simetri antar neuron. Xavier dan He dipakai sebagai dua baseline yang memang dirancang supaya sinyal dan gradien tetap berada pada skala yang wajar di awal training. Hasilnya cukup jelas. Pada Gambar 6 dan Tabel 1, inisialisasi `zero` tertinggal jauh dari Xavier dan He. Xavier memberi hasil terbaik sedangkan He tetap jauh lebih baik daripada `zero`. Jadi, eksperimen ini menunjukkan bahwa inisialisasi bukan sekadar detail kecil. Nilai awal bobot benar-benar bisa memengaruhi apakah proses optimisasi berjalan lancar atau tidak.

4.6 Pengaruh Regularisasi

Regularisasi diuji dalam tiga bentuk: tanpa regularisasi, L1 lemah, dan L2 lemah [17, 18]. Nilai koefisiennya dipilih kecil supaya efeknya tetap bisa diamati tanpa terlalu mendominasi objective utama. Pada eksperimen ini, pengaruh regularisasi terhadap akurasi akhir memang tidak besar. Pada run terbaru, `NoReg` memberi akurasi tertinggi 0.7480 sedangkan L2 memberi *validation loss* terendah 0.491365. Jadi, pada model dengan ukuran sedang seperti ini, regularisasi lebih terasa sebagai alat bantu stabilisasi daripada sumber utama peningkatan akurasi.



Gambar 8: Eksperimen regularisasi: panel kiri memperlihatkan distribusi bobot akhir, sedangkan panel kanan memperlihatkan distribusi gradien akhir setelah pelatihan.



Gambar 9: Eksperimen RMSNorm: panel kiri menunjukkan kurva loss train/validasi untuk model dengan dan tanpa RMSNorm, sedangkan panel kanan merangkum trade-off akhir antara validation loss dan validation accuracy serta selisih prediksi antar model.

4.7 Pengaruh Normalisasi RMSNorm

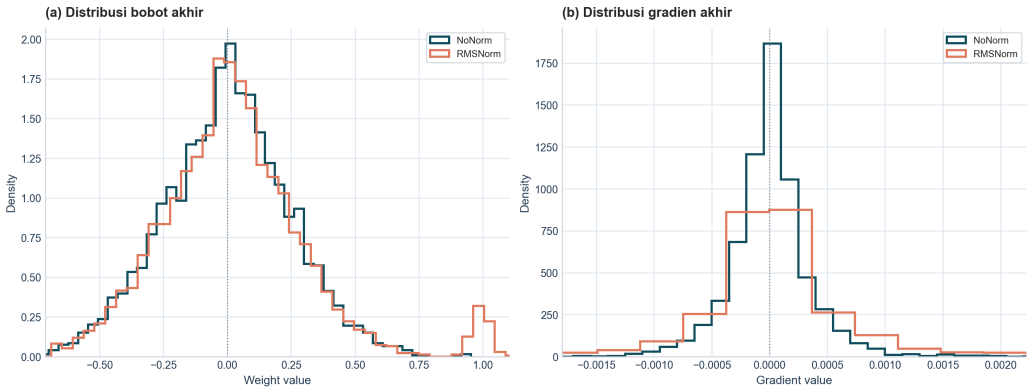
Eksperimen RMSNorm dibuat dengan arsitektur yang sama. Bedanya, satu model memakai RMSNorm setelah setiap transformasi linear tersembunyi sedangkan model pembanding tidak memakainya [3, 9]. Dengan cara ini, pengaruh RMSNorm bisa dilihat dengan lebih jelas. Hasil pada Gambar 9 dan Gambar 10 menunjukkan bahwa RMSNorm dapat menaikkan akurasi walaupun *validation loss* akhir tidak selalu ikut turun. Artinya, normalisasi ini tetap bisa membantu model membuat keputusan klasifikasi yang lebih baik pada beberapa kasus.

4.8 Perbandingan dengan `sklearn` MLP

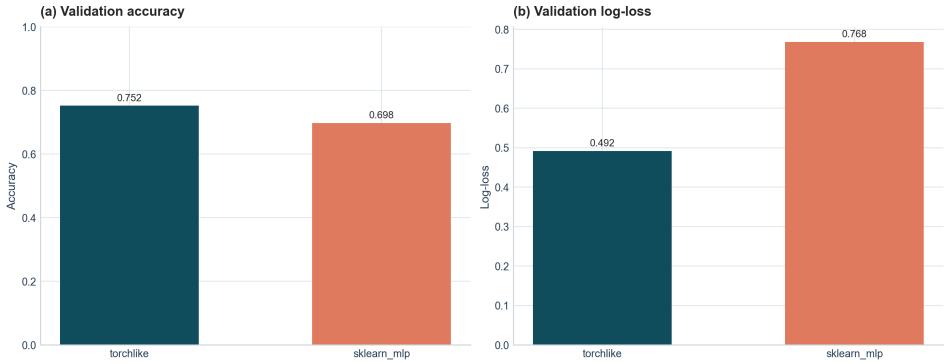
Sebagai baseline eksternal, model `sklearn MLPClassifier` dilatih dengan konfigurasi yang dibuat semirip mungkin: tiga hidden layer berukuran 32, aktivasi `tanh`, optimizer keluarga SGD, *learning rate* 0.03, regularisasi 10^{-4}, batch size 256, horizon 160 epoch, dan seed 123 [21, 25]. Aktivasi `tanh` dipilih karena tersedia secara native di kedua framework sehingga perbandingannya lebih adil. Hasil pada Gambar 11 menunjukkan bahwa implementasi `torchlike` memberi metrik validasi yang lebih baik pada setup ini, yaitu akurasi 0.7520 dan log-loss 0.491776 dibanding `sklearn_mlp` yang mencapai 0.6975 dan 0.768242. Ini menunjukkan bahwa implementasi *from scratch* yang kami buat tetap bisa bersaing dengan baseline dari library yang sudah mapan.

4.9 Tambahan: SGD vs Adam

Perbandingan optimizer dibuat pada arsitektur dasar yang sama agar yang diuji memang benar-benar cara update parameternya. *Learning rate* antara SGD dan Adam tidak disamakan angkanya karena



Gambar 10: Eksperimen RMSNorm: panel kiri memperlihatkan distribusi bobot akhir, sedangkan panel kanan memperlihatkan distribusi gradien akhir untuk model baseline dan model dengan RMSNorm.



Gambar 11: Perbandingan `torchlike` dan `sklearn`: panel kiri menunjukkan validation accuracy, sedangkan panel kanan menunjukkan validation log-loss.

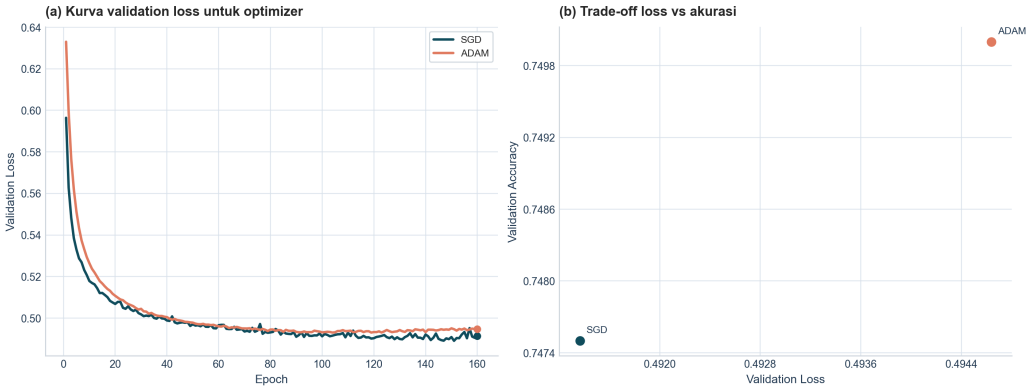
keduanya memang punya karakter langkah yang berbeda. Kalau dipaksa sama, hasilnya justru bisa bias. Karena itu, studi ini membandingkan konfigurasi yang stabil untuk masing-masing optimizer, yaitu SGD dengan 0.03 dan Adam dengan 2×10^{-4}. Hasil pada Gambar 12 menunjukkan bahwa Adam mencapai epoch terbaik lebih cepat, yaitu pada epoch 104, sedangkan SGD baru mencapai titik terbaik pada epoch 151. Namun, SGD justru memberi best loss yang lebih rendah, yaitu 0.489066, sementara Adam memberi akurasi akhir sedikit lebih tinggi, yaitu 0.7500. Jadi, pada run ini Adam unggul dari sisi kecepatan menuju titik terbaik, sedangkan SGD masih kompetitif dari sisi loss terbaik.

Tabel 1: Ringkasan metrik eksperimen (compact view).

Kategori	Konfigurasi	Val Acc	Val Loss
Depth/Width	width=8 (depth=3)	0.7520	0.488768
	width=16 (depth=3)	0.7500	0.492507
	width=24 (depth=3)	0.7520	0.495772
	width=32 (depth=3)	0.7475	0.491365
	depth=2 (width=32)	0.7480	0.494147
	depth=3 (width=32)	0.7475	0.491365
	depth=4 (width=32)	0.7500	0.503533
Aktivasi	ReLU	0.7505	0.497548
	Sigmoid	0.7535	0.490337
	Tanh	0.7520	0.491776
	GELU	0.7500	0.490526
	SELU	0.7535	0.498220
Learning Rate	lr=0.005	0.7490	0.503617
	lr=0.03	0.7475	0.491365
	lr=0.10	0.7370	0.552049
Inisialisasi	Zero	0.6155	0.666224
	Xavier	0.7530	0.488897
	He	0.7475	0.491365
Regularisasi	NoReg	0.7480	0.491603
	L1	0.7475	0.491568
	L2	0.7475	0.491365

Tabel 2: Komparasi model dan optimizer.

Eksperimen	Konfigurasi	Val Acc	Val Loss	Catatan
RMSNorm	SGD-NoNorm	0.7395	0.499106	baseline
	SGD-RMSNorm	0.7480	0.508377	pred diff 4.05%
Sklearn	torchlike	0.7520	0.491776	log-loss
	sklearn_MLP	0.6975	0.768242	log-loss
Optimizer	SGD	0.7475	0.491365	best 0.489066@ep151
	Adam	0.7500	0.494638	best 0.493030@ep104



Gambar 12: Perbandingan optimizer: panel kiri menunjukkan kurva validation loss untuk SGD dan Adam, sedangkan panel kanan merangkum trade-off akhir antara validation loss dan validation accuracy.

Tabel 3: Tabel kontribusi anggota kelompok `TorchIsAllYouNeed`.

Nama	NIM	Kontribusi
Shanice Feodora Tjahjono	13523097	Linear, Activation Functions, Experimental Design
Muhammad Fathur Rizky	13523105	SGD, Adam, Automatic Differentiation, RMSNorm
Ahmad Wicaksono	13523121	Loss Functions, Experimental Design

5 Kesimpulan dan Saran

Implementasi `torchlike` berhasil membangun pipeline pelatihan FFNN secara end-to-end mulai dari autodiff, fungsi aktivasi, inisialisasi bobot, optimizer, regularisasi, sampai RMSNorm. Dari eksperimen yang dilakukan, terlihat bahwa kapasitas model yang lebih besar tidak selalu memberi generalisasi yang lebih baik. Pilihan aktivasi dan inisialisasi juga cukup berpengaruh ke jalannya optimisasi, sedangkan RMSNorm pada beberapa kondisi bisa membantu hasil klasifikasi. Secara khusus, eksperimen inisialisasi menunjukkan bahwa `zero` bukan pilihan awal yang baik karena gagal memecah simetri, sedangkan Xavier dan He jauh lebih stabil. Untuk pengembangan selanjutnya, eksperimen bisa diulang pada beberapa seed dan ditambah metrik lain seperti *confusion matrix* atau analisis kalibrasi probabilitas.

6 Pembagian Tugas

Pembagian tugas anggota kelompok dirangkum secara eksplisit pada Tabel 3. Ringkasan ini memetakan setiap anggota ke komponen implementasi dan rancangan eksperimen yang menjadi tanggung jawab utamanya.

7 Lampiran

7.1 Repositori Proyek

Kode sumber implementasi, eksperimen, dan artefak pendukung laporan tersedia pada:

`https://github.com/fathurwithyou/TorchIsAllYouNeed`

Referensi

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[2] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[3] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[4] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In *IEEE International Conference on Computer Vision (ICCV)*, 2015.

[6] Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[7] Gunter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-Normalizing Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Vinod Nair and Geoffrey E. Hinton. Rectified Linear Units Improve Restricted Boltzmann Machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010.

[9] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer Normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[10] Robert E. Wengert. A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8):463–464, 1964.

[11] Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16(2):146–160, 1976.

[12] Paul J. Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, 1974.

[13] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[14] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic Differentiation in Machine Learning: a Survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[15] Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[16] Leon Bottou. Stochastic Gradient Descent Tricks. In Grégoire Montavon, Genevieve Orr, and Klaus-Robert Muller, editors, *Neural Networks: Tricks of the Trade*, pages 421–436. Springer, 2012.

[17] Robert Tibshirani. Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.

[18] Arthur E. Hoerl and Robert W. Kennard. Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1):55–67, 1970.

[19] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[20] Adam Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[21] Fabian Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[22] PyTorch Contributors. BCEWithLogitsLoss documentation. PyTorch documentation. Available at `https://pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html`. Accessed March 6, 2026.

[23] scikit-learn Contributors. StandardScaler documentation. scikit-learn documentation. Available at `https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html`. Accessed March 6, 2026.

[24] scikit-learn Contributors. OneHotEncoder documentation. scikit-learn documentation. Available at `https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html`. Accessed March 6, 2026.

[25] scikit-learn Contributors. MLPClassifier documentation. scikit-learn documentation. Available at `https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html`. Accessed March 6, 2026.

[26] Andrej Karpathy. The spelled-out intro to neural networks and backpropagation: building micrograd. Video lecture and accompanying educational implementation, 2022.

[27] Peter L. Bartlett, Michael I. Jordan, and Jon D. McAuliffe. Convexity, Classification, and Risk Bounds. *Journal of the American Statistical Association*, 101(473):138–156, 2006.